

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

TỪ MINH PHƯƠNG

GIÁO TRÌNH

Hệ điều hành

Hà nội 2013

LỜI NÓI ĐẦU

Hệ điều hành là thành phần quan trọng trong máy tính. Nắm vững kiến thức về hệ điều hành là cơ sở cho việc hiểu biết sâu sắc hệ thống máy tính nói chung. Chính vì vậy, kiến thức về hệ điều hành là phần kiến thức bắt buộc đối với chuyên gia công nghệ thông tin và các ngành liên quan.

Môn học Hệ điều hành là môn học cơ sở trong chương trình đào tạo đại học, cao đẳng ngành công nghệ thông tin. Kiến thức liên quan tới hệ điều hành bao gồm ba dạng chính. Thứ nhất, đó là kiến thức và kỹ năng về việc cài đặt, sử dụng, khai thác, đánh giá hệ điều hành một cách hiệu quả. Các kiến thức này rất cần thiết cho người sử dụng cũng như những chuyên gia về vận hành, phục vụ hạ tầng tính toán nói chung. Thứ hai, hệ điều hành được xem xét từ góc độ thiết kế và xây dựng. Đây là những kiến thức cần thiết cho chuyên gia về hệ thống hoặc những người sẽ tham gia thiết kế, xây dựng hệ điều hành. Thứ ba, đó là kiến thức về các khái niệm và nguyên lý chung về hệ điều hành như một thành phần quan trọng của hệ thống máy tính. Cần lưu ý rằng việc phân chia này là tương đối và các khối kiến thức có liên quan đến nhau.

Trong tài liệu này, kiến thức về hệ điều hành được trình bày theo dạng thứ ba với mục đích cung cấp kiến thức về các khái niệm và nguyên lý hoạt động của hệ điều hành, từ đây giúp người đọc có hiểu biết sâu hơn về hệ thống máy tính. Những nguyên lý và khái niệm trình bày trong tài liệu mang tính tổng quát cho hệ điều hành nói chung, thay vì dựa trên một hệ điều hành cụ thể. Tuy nhiên, để giúp người đọc có được liên kết giữa lý thuyết và thực tế, một số kỹ thuật trong hệ điều hành cụ thể sẽ được trình bày như những ví dụ minh họa, đặc biệt chú ý tới những hệ điều hành được sử dụng rộng rãi tại Việt Nam.

Các nội dung của giáo trình được trình bày thành bốn chương.

Chương 1 bao gồm những khái niệm chung về hệ điều hành, vai trò trong hệ thống máy tính, các thành phần chức năng và một số kiểu kiến trúc thông dụng. Chương 1 cũng tóm tắt quá trình hình thành và phát triển hệ điều hành, qua đó trình bày một số khái niệm và kỹ thuật quan trọng trong hệ điều hành hiện nay cũng như một số xu hướng tính toán mới hình thành. Kết thúc chương là ví dụ một số hệ điều hành tiêu biểu.

Chương 2 trình bày về quản lý tiến trình trong hệ điều hành, tập trung vào quản lý tiến trình trong hệ thống với một CPU và nhiều tiến trình. Những nội dung chính của chương bao gồm: khái niệm tiến trình, trạng thái tiến trình, các thao tác và thông tin quản lý tiến trình, dòng thực hiện, vấn đề điều độ tiến trình, đồng bộ hóa các tiến trình đồng thời.

Chương 3 trình bày về quản lý bộ nhớ. Nội dung chính của chương 3 bao gồm: các vấn đề liên quan tới bộ nhớ và địa chỉ, một số kỹ thuật tổ chức chương trình, kỹ thuật phân chương, phân trang, phân đoạn bộ nhớ, khái niệm và cách tổ chức quản lý bộ nhớ ảo. Những khái niệm lý thuyết trình bày trong chương được minh họa qua hai ví dụ: hỗ trợ quản lý bộ nhớ trong vi xử lý Intel Pentium, và quản lý bộ nhớ trong hệ điều hành Windows XP.

Chương 4 trình bày về hệ thống file với những nội dung chính sau: khái niệm file và thư mục, các thao tác với file và thư mục, tổ chức bên trong của file và thư mục, vấn đề cấp phát

và quản lý không gian lưu trữ của file, các vấn đề về độ tin cậy và an toàn bảo mật của hệ thống file. Chương 4 cũng bao gồm một số khái niệm quan trọng về tổ chức vào ra của máy tính và phân hệ quản lý vào/ra của hệ điều hành, trong đó trọng tâm là vào/ra với đĩa cứng máy tính.

Tài liệu được biên soạn từ kinh nghiệm giảng dạy học phần Hệ điều hành tại Học viện Công nghệ bưu chính viễn thông, trên cơ sở tiếp thu phản hồi từ sinh viên và đồng nghiệp của tác giả. Tài liệu có thể sử dụng làm tài liệu học tập cho sinh viên đại học, cao đẳng ngành công nghệ thông tin và các ngành liên quan, ngoài ra có thể sử dụng với mục đích tham khảo cho những người quan tâm tới hệ điều hành và hệ thống máy tính.

Trong quá trình biên soạn tài liệu, mặc dù tác giả đã có nhiều cố gắng song không thể tránh khỏi những thiếu sót. Ngoài ra, hệ điều hành là một lĩnh vực có nhiều thay đổi của khoa học máy tính đòi hỏi tài liệu về hệ điều hành phải được cập nhật thường xuyên. Tác giả rất mong muốn nhận được ý kiến phản hồi, góp ý cho các thiếu sót cũng như ý kiến về việc cập nhật, hoàn thiện nội dung của tài liệu.

Hà nội 12/2013

TÁC GIẢ

PTIT

MỤC LỤC

CHƯƠNG 1: GIỚI THIỆU CHUNG	8
1.1. CÁC THÀNH PHẦN CỦA HỆ THỐNG MÁY TÍNH.....	8
1.2. TỔ CHỨC PHẦN CỨNG CỦA MÁY TÍNH	9
1.3. KHÁI NIỆM HỆ ĐIỀU HÀNH	12
1.4. CÁC DỊCH VỤ DO HỆ ĐIỀU HÀNH CUNG CẤP	15
1.5. GIAO DIỆN LẬP TRÌNH CỦA HỆ ĐIỀU HÀNH	17
1.6. QUÁ TRÌNH PHÁT TRIỂN VÀ MỘT SỐ KHÁI NIỆM QUAN TRỌNG	19
1.7. CẤU TRÚC HỆ ĐIỀU HÀNH	24
1.7.1. Các thành phần của hệ điều hành	24
1.7.2. Nhân của hệ điều hành	28
1.7.3. Một số kiểu cấu trúc hệ điều hành	29
1.8. MỘT SỐ HỆ ĐIỀU HÀNH CỤ THỂ.....	34
1.9. CÂU HỎI VÀ BÀI TẬP CHƯƠNG	37
CHƯƠNG 2: QUẢN LÝ TIẾN TRÌNH.....	39
2.1. CÁC KHÁI NIỆM LIÊN QUAN ĐẾN TIẾN TRÌNH.....	39
2.1.1. Tiến trình là gì.....	39
2.1.2. Trạng thái của tiến trình.....	40
2.1.3. Thông tin mô tả tiến trình.....	42
2.1.4. Bảng và danh sách tiến trình.....	43
2.1.5. Các thao tác với tiến trình.....	44
2.2. LUỒNG.....	47
2.2.1. Luồng thực hiện là gì.....	47
2.2.2. Ví dụ đa luồng trên hệ thống cụ thể	48
2.2.3. Tài nguyên của tiến trình và luồng.....	52
2.2.4. Ưu điểm của mô hình đa luồng.....	53
2.2.5. Luồng mức người dùng và luồng mức nhân.....	53
2.3. ĐIỀU ĐỘ TIẾN TRÌNH.....	56
2.3.1. Khái niệm điều độ	56
2.3.2. Các dạng điều độ.....	56
2.3.3. Các tiêu chí điều độ	58
2.3.4. Các thuật toán điều độ	59
2.3.5. Điều độ trên hệ thống cụ thể.....	64
2.4. ĐỒNG BỘ HÓA TIẾN TRÌNH ĐỒNG THỜI.....	66
2.4.1. Các vấn đề đối với tiến trình đồng thời	66

2.4.2. Yêu cầu với giải pháp cho đoạn nguy hiểm.....	68
2.4.3. Giải thuật Peterson	69
2.4.4. Giải pháp phần cứng.....	70
2.4.5. Cờ hiệu (semaphore)	72
2.4.6. Một số bài toán đồng bộ	74
2.4.7. Monitor	77
2.4.8. Bể tắc	80
2.5. CÂU HỎI VÀ BÀI TẬP CHƯƠNG	88
CHƯƠNG 3: QUẢN LÝ BỘ NHỚ.....	91
3.1. ĐỊA CHỈ VÀ CÁC VẤN ĐỀ LIÊN QUAN	91
3.1.1. Vấn đề gán địa chỉ.....	91
3.1.2. Địa chỉ lô gic và địa chỉ vật lý	93
3.2. MỘT SỐ CÁCH TỔ CHỨC CHƯƠNG TRÌNH.....	93
3.2.1. Tải trong quá trình thực hiện	93
3.2.2. Liên kết động và thư viện dùng chung.....	94
3.3. PHÂN CHƯƠNG BỘ NHỚ	95
3.3.1. Phân chương cố định	96
3.3.2. Phân chương động.....	98
3.3.3. Phương pháp kề cận	100
3.3.4. Ánh xạ địa chỉ và chống truy cập trái phép.....	101
3.3.5. Trao đổi giữa bộ nhớ và đĩa (swapping).....	102
3.4. PHÂN TRANG BỘ NHỚ.....	103
3.4.1. Khái niệm phân trang bộ nhớ.....	103
3.4.2. Ánh xạ địa chỉ	104
3.4.3. Tổ chức bảng phân trang	107
3.5. PHÂN ĐOẠN BỘ NHỚ	112
3.5.1 Khái niệm.....	112
3.5.2. Ánh xạ địa chỉ và chống truy cập trái phép	112
3.5.3. Kết hợp phân đoạn với phân trang	114
3.6. BỘ NHỚ ẢO	114
3.6.1. Khái niệm bộ nhớ ảo	114
3.6.2. Nạp trang theo nhu cầu.....	115
3.7. ĐỔI TRANG	118
3.7.1. Tại sao phải đổi trang	118
3.7.2. Các chiến lược đổi trang.....	119
3.8. CẤP PHÁT KHUNG TRANG.....	125

3.8.1. Giới hạn số lượng khung	125
3.8.2. Phạm vi cấp phát khung.....	126
3.9. TÌNH TRẠNG TRÌ TRỆ	126
3.10. QUẢN LÝ BỘ NHỚ TRONG INTEL PENTIUM.....	127
3.11. QUẢN LÝ BỘ NHỚ TRONG WINDOWS 32 bit	130
3.12. CÂU HỎI VÀ BÀI TẬP CHƯƠNG	131
CHƯƠNG 4: HỆ THỐNG FILE.....	133
4.1. KHÁI NIỆM FILE.....	133
4.1.1. File là gì ?	133
4.1.2. Thuộc tính của file.....	134
4.1.3. Cấu trúc file	136
4.2. CÁC PHƯƠNG PHÁP TRUY CẬP FILE	137
4.2.1. Truy cập tuần tự	137
4.2.2. Truy cập trực tiếp	137
4.2.3. Truy cập dựa trên chỉ số	138
4.3. CÁC THAO TÁC VỚI FILE	139
4.4. THƯ MỤC	141
4.4.1. Khái niệm thư mục	141
4.4.2. Các thao tác với thư mục	142
4.4.3. Cấu trúc hệ thống thư mục.....	143
4.4.4. Tên đường dẫn	147
4.5. CẤP PHÁT KHÔNG GIAN CHO FILE	148
4.5.1. Cấp phát các khối liên tiếp.....	148
4.5.2. Sử dụng danh sách kết nối	150
4.5.3. Sử dụng danh sách kết nối trên bảng chỉ số.....	151
4.5.4. Sử dụng khối chỉ số	152
4.6. QUẢN LÝ KHÔNG GIAN TRÊN ĐĨA.....	153
4.6.1. Kích thước khối.....	153
4.6.2. Quản lý các khối trống	154
4.7. TỔ CHỨC BÊN TRONG CỦA THƯ MỤC	156
4.7.1. Danh sách.....	156
4.7.2. Cây nhị phân	156
4.7.3. Bảng băm	156
4.7.4. Tổ chức thư mục của DOS (FAT).....	157
4.7.5. Thư mục của Linux	157
4.8. ĐỘ TIN CẬY CỦA HỆ THỐNG FILE	158

4.8.1. Phát hiện và loại trừ các khối hỏng	158
4.8.2. Sao dự phòng	159
4.9. BẢO MẬT CHO HỆ THỐNG FILE	161
4.9.1. Sử dụng mật khẩu	161
4.9.2. Danh sách quản lý truy cập	162
4.10. HỆ THỐNG FILE FAT	163
4.10.1. Đĩa logic	164
4.10.2. Boot sector	165
4.10.3. Bảng FAT	167
4.10.4. Thư mục gốc	168
4.11. TỔ CHỨC THÔNG TIN TRÊN BỘ NHỚ THỨ CẤP	169
4.11.1. Tổ chức đĩa cứng	169
4.11.2. Điều độ đĩa	172
4.12. QUẢN LÝ VÀO/RA	176
4.12.1. Phần cứng	176
4.12.2. Tổ chức phân hệ quản lý vào/ra	177
4.12.3. Quản lý vào/ra mức trên	179
4.13. CÂU HỎI VÀ BÀI TẬP CHƯƠNG	181
TÀI LIỆU THAM KHẢO	183

CHƯƠNG 1: GIỚI THIỆU CHUNG

1.1. CÁC THÀNH PHẦN CỦA HỆ THỐNG MÁY TÍNH

Một hệ thống máy tính nói chung có thể phân chia sơ bộ thành phần cứng và phần mềm. Phần cứng cung cấp các tài nguyên cần thiết cho việc tính toán, xử lý dữ liệu. Phần mềm gồm các chương trình quy định cụ thể việc xử lý đó. Để thực hiện công việc tính toán hoặc xử lý dữ liệu cụ thể cần có các chương trình gọi là chương trình ứng dụng. Có thể kể một số chương trình ứng dụng thường gặp như chương trình soạn thảo văn bản, chương trình trò chơi, hệ quản trị cơ sở dữ liệu, chương trình truyền thông .v.v.

Phần cứng có thể biểu diễn như lớp dưới cùng, là cơ sở của toàn hệ thống. Đây là những thiết bị cụ thể như CPU, bộ nhớ, thiết bị nhớ ngoài, thiết bị vào ra. Chương trình ứng dụng là lớp trên của hệ thống, là phần mà người dùng xây dựng nên và tương tác trong quá trình giải quyết các nhiệm vụ của mình. (Khái niệm người dùng ở đây bao gồm cả người sử dụng thuần túy lẫn người viết ra các chương trình ứng dụng)

Ngoài phần cứng và trình ứng dụng, hệ thống máy tính còn có một thành phần quan trọng là hệ điều hành. Hệ điều hành là phần mềm đóng vai trò trung gian giữa phần cứng và người sử dụng cùng các chương trình ứng dụng của họ. Nhiệm vụ của hệ điều hành là làm cho việc sử dụng hệ thống máy tính được *tiện lợi* và *hiệu quả*. Các chương trình ứng dụng khi chạy đều cần thực hiện một số thao tác chung như điều khiển thiết bị vào ra. Những thao tác phân phối và điều khiển tài nguyên chung như vậy sẽ được gộp chung lại trong phạm vi hệ điều hành. Nhờ có hệ điều hành, người dùng có thể dễ dàng tương tác và sử dụng máy tính, ví dụ bằng cách thông qua các giao diện đồ họa. Cũng nhờ có hệ điều hành, phần cứng của máy tính được quản lý và sử dụng hiệu quả hơn.

Người sử dụng
Chương trình ứng dụng, chương trình hệ thống và tiện ích
Hệ điều hành
Phần cứng

Hình 1.1: Các thành phần của hệ thống máy tính

Ngoài chương trình ứng dụng và hệ điều hành còn có các chương trình hệ thống và chương trình tiện ích. Đây là những chương trình không thuộc phần cốt lõi của hệ điều hành

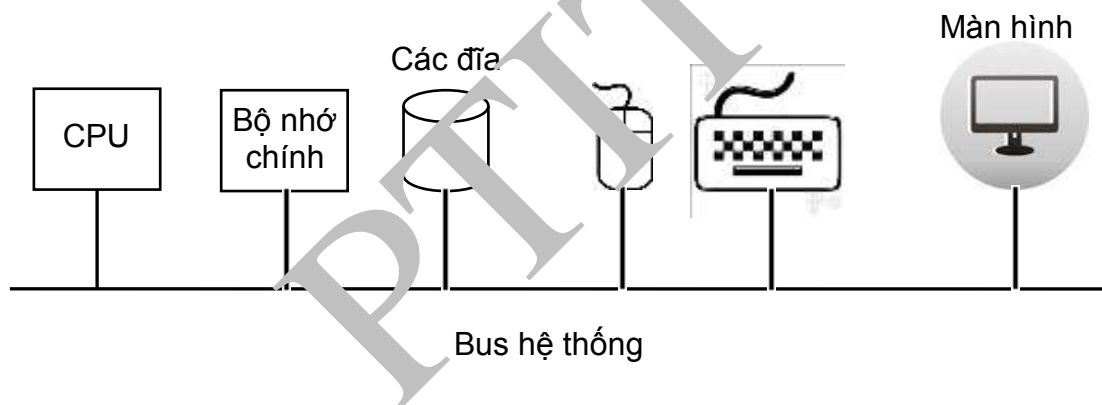
nhưng được xây dựng để thực hiện những thao tác thường diễn ra trong hệ thống hoặc giúp người dùng thực hiện một số công việc dễ dàng hơn.

Các thành phần của hệ thống máy tính được thể hiện trên hình 1.1, trong đó phần cứng là lớp dưới cùng và người dùng giao tiếp với trình ứng dụng là thành phần trên cùng của hệ thống.

1.2. TỔ CHỨC PHẦN CỨNG CỦA MÁY TÍNH

Hệ điều hành giao tiếp trực tiếp với phần cứng máy tính và quản lý các tài nguyên phần cứng. Các khái niệm về tổ chức phần cứng rất quan trọng và cần thiết cho việc tìm hiểu về hệ điều hành. Để thuận lợi cho việc trình bày về hệ điều hành, phần này sẽ tóm tắt một số nội dung liên quan về tổ chức và kiến trúc phần cứng của máy tính.

Kiến trúc chung. Máy tính bao gồm một hoặc nhiều CPU (khối xử lý trung tâm), bộ nhớ chính, các đĩa từ và thiết bị nhớ SSD (còn gọi là đĩa điện tử), màn hình, các thiết bị vào ra dữ liệu khác như chuột, bàn phím, máy in, màn cảm ứng, micro, loa, ... Các bộ phận này được kết nối trao đổi thông tin với nhau thông qua bus hệ thống như minh họa trên hình 1.2.



Hình 1.2. Các thành phần của phần cứng máy tính

CPU và quy trình thực hiện lệnh. CPU (khối xử lý trung tâm) là thành phần quan trọng nhất của hệ thống máy tính. CPU bao gồm khối ALU thực hiện các phép toán số học và logic, khối điều khiển thực hiện việc giải mã lệnh và điều khiển hoạt động chung. Ngoài ra còn có các thanh ghi, thực chất là bộ nhớ của CPU dùng để lưu các dữ liệu tạm thời và các thông tin về trạng thái của CPU và toàn hệ thống.

Nhiệm vụ chủ yếu của CPU là thực hiện các chương trình. Mỗi chương trình là một tập hợp các lệnh để chỉ thị cho CPU biết cần làm gì. Khi chương trình được thực hiện, các lệnh của chương trình được đọc vào và lưu trong bộ nhớ chính. CPU lần lượt đọc từng lệnh từ bộ nhớ chính và thực hiện lệnh. Tùy vào lệnh cụ thể, việc thực hiện một lệnh có thể dẫn tới yêu cầu có thêm các thao tác đọc hoặc ghi bộ nhớ khác. Ví dụ lệnh tăng giá trị một biến đòi hỏi thao tác đọc giá trị biến đó từ bộ nhớ, sau đó ghi giá trị mới ra bộ nhớ.

Ngắt. Quá trình thực hiện các lệnh của chương trình diễn ra tuần tự, sau khi xong một lệnh CPU sẽ thực hiện lệnh tiếp theo, trừ khi có các lệnh rẽ nhánh hay vòng lặp. Tuy nhiên, trong hệ thống máy tính còn có các sự kiện xảy ra và cần được CPU xử lý kịp thời. Ví dụ, thiết bị phần cứng có thể gửi tín hiệu để thông báo xảy ra sự kiện cần xử lý, như khi người dùng bấm bàn phím. Việc xử lý sự kiện các sự kiện như vậy được thực hiện nhờ cơ chế *ngắt* (interrupt). Thiết bị phần cứng có thể yêu cầu thực hiện ngắt bằng cách gửi tín hiệu qua bus. Phần mềm, tức là chương trình đang thực hiện, cũng có thể yêu cầu ngắt bằng cách sử dụng lời gọi hệ thống (system call). Chẳng hạn khi cần ghi ra file, chương trình có thể gửi yêu cầu ngắt dưới dạng lời gọi hệ thống ghi ra file. Hệ thống sẽ chuyển sang xử lý ngắt trước khi quay lại thực hiện tiếp chương trình theo thứ tự thông thường.

Xử lý ngắt. Khi có ngắt, CPU sẽ tạm dừng công việc đang thực hiện và chuyển sang thực hiện hàm xử lý ngắt. Sau khi thực hiện xong hàm xử lý ngắt, hệ thống sẽ quay lại điểm tạm dừng và thực hiện tiếp công việc bị ngắt. Cơ chế xử lý ngắt cụ thể phụ thuộc vào từng dòng máy tính và hệ điều hành, tuy nhiên thông thường các máy tính sử dụng cơ chế xử lý ngắt như sau. Các hàm xử lý ngắt được lưu trong bộ nhớ. Các hàm xử lý ngắt do phần cứng đảm nhiệm được lưu trong bộ nhớ ROM hoặc EPROM như một thành phần của phần cứng, ví dụ như một thành phần của BIOS trên PC. Hàm xử lý ngắt của hệ điều hành được tải vào và lưu trong bộ nhớ RAM. Địa chỉ các hàm xử lý ngắt được lưu trong một mảng gọi là vec tơ ngắt, nằm ở phần địa chỉ thấp của bộ nhớ, bắt đầu từ địa chỉ 0. Mỗi phần tử của vec tơ ngắt có kích thước cố định và chứa con trỏ tới hàm xử lý ngắt tương ứng. Như vậy, ví dụ khi xuất hiện ngắt có số thứ tự bằng 2, CPU sẽ đọc nội dung ô thứ 2 của vec tơ ngắt để có địa chỉ hàm xử lý ngắt, sau đó chuyển tới địa chỉ đó để thực hiện hàm xử lý ngắt. Các hệ điều hành thông dụng như Windows, Linux xử lý ngắt theo quy trình này.

Trong các hệ điều hành đa chương trình, tức là hệ điều hành cho phép nhiều tiến trình được thực hiện đồng thời, hệ điều hành thường sử dụng ngắt từ bộ định thời timer để thu hồi quyền điều khiển CPU từ một chương trình đang thực hiện để phân phối cho chương trình khác. Timer là một cơ chế phần cứng cho phép sinh ra ngắt sau một khoảng thời gian do hệ điều hành quy định. Ngắt này được chuyển cho hàm xử lý ngắt của hệ điều hành xử lý, thường là để phân phối lại quyền sử dụng CPU.

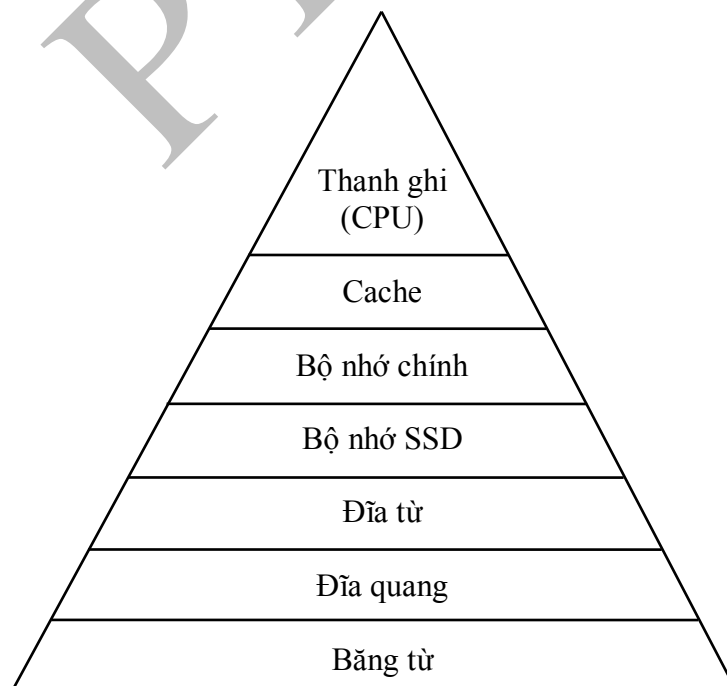
Bộ nhớ chính. Bộ nhớ chính là nơi chứa các chương trình đang được thực hiện, bao gồm cả các lệnh của chương trình cũng như dữ liệu. Bộ nhớ chính là dạng bộ nhớ cho phép đọc và ghi theo thứ tự bất kỳ, gọi là bộ nhớ truy cập ngẫu nhiên (Random Access Memory - RAM), do vậy thường được gọi tắt là RAM. Cần lưu ý rằng, máy tính có một số dạng bộ nhớ khác, ví dụ như bộ nhớ chỉ cho phép đọc (ROM), hoặc bộ nhớ chỉ cho phép ghi với thiết bị ghi đặc biệt (EPROM), và các loại bộ nhớ này có thể kết hợp với RAM để tạo thành bộ nhớ chính của máy tính.

Bộ nhớ máy tính có thể coi như một mảng các ô nhớ được đánh địa chỉ. Bộ nhớ thường được truy cập theo đơn vị là byte (B), mỗi byte gồm 8 bit (b). Một số máy tính sử dụng đơn vị bộ nhớ là từ (word), mỗi từ có thể có kích thước bằng 2, 4, 8 byte; tuy nhiên, kiểu đơn vị bộ nhớ này ít thông dụng trong các máy tính hiện nay. Như vậy, mỗi byte của bộ nhớ được coi là một ô nhớ và được truy cập theo địa chỉ của byte.

Để tính các lượng bộ nhớ lớn hơn, các đơn vị là lũy thừa bậc 2 của byte thường được sử dụng như: kilobyte (KB) = 2^{10} byte, megabyte (MB) = 2^{20} byte, gigabyte (GB) = 2^{30} byte, terabyte (TB) = 2^{40} byte, petabyte (PB) = 2^{50} byte, hexabyte (HB) = 2^{60} byte. Cách tổ chức và đơn vị bộ nhớ như vậy cũng được sử dụng cho các dạng bộ nhớ khác như bộ nhớ ngoài của máy tính.

Tổ chức hệ thống bộ nhớ. Ngoài bộ nhớ chính (RAM), máy tính còn nhiều dạng bộ nhớ khác như bộ nhớ thanh ghi, bộ nhớ trên đĩa... Lý do phải sử dụng nhiều dạng bộ nhớ là do không có dạng thiết bị nhớ nào thỏa mãn đồng thời các yêu cầu đặt ra về lưu trữ thông tin. Bộ nhớ máy tính lý tưởng là bộ nhớ thỏa mãn đồng thời các yêu cầu chính sau: 1) dung lượng lớn; 2) tốc độ truy cập nhanh; 3) giá thành thấp; 4) có khả năng lưu trữ lâu bền cả khi có điện và không có điện. Những yêu cầu này là mâu thuẫn với nhau, chẳng hạn thiết bị nhớ tốc độ cao có giá cao và không lưu được thông tin khi không có điện. Cụ thể, bộ nhớ chính, được xây dựng dựa trên công nghệ DRAM (dynamic random-access memory), mặc dù có tốc độ truy cập tương đối cao xong không đủ lớn để lưu trữ thường xuyên tất cả chương trình và dữ liệu. Ngoài ra, nội dung bộ nhớ sử dụng DRAM sẽ bị xóa khi tắt nguồn, do vậy không phù hợp để lưu trữ lâu thông tin lâu dài.

Để giải quyết vấn đề nói trên, hệ thống bộ nhớ trong máy tính được tạo thành từ nhiều dạng bộ nhớ khác nhau, mỗi dạng có ưu điểm về một mặt nào đó như tốc độ, dung lượng, giá thành. Hệ thống bộ nhớ được tổ chức như một cấu trúc phân cấp hình tháp như minh họa trên hình 1.3, trong đó các dạng ở nhớ ở mức trên có tốc độ và giá thành cao, do vậy chỉ có thể sử dụng với dung lượng nhỏ. Ngược lại, phía dưới của tháp là bộ nhớ dung lượng lớn và rẻ nhưng chậm.



Hình 1.3. Tổ chức phân cấp dạng hình tháp của hệ thống bộ nhớ máy tính

Giới thiệu chung

Các dạng bộ nhớ từ SSD trở xuống có thể lưu trữ thông tin và dữ liệu ngay cả khi không có điện, trong khi các dạng bộ nhớ phía trên trong sơ đồ phân cấp bị mất nội dung khi không có nguồn nuôi. Bộ nhớ SSD là dạng bộ nhớ sử dụng công nghệ nhớ mới, có tốc độ nhanh hơn đĩa từ, trong khi vẫn có thể lưu thông tin khi không có điện. Dạng bộ nhớ SSD thông dụng nhất là dạng bộ nhớ flash dùng cho các thiết bị như máy ảnh, điện thoại di động thông minh. Một số máy tính xách tay (laptop) mới cũng sử dụng bộ nhớ loại này kết hợp với đĩa cứng, trong đó bộ nhớ SSD được sử dụng cho để lưu những thông tin cần truy cập nhanh như thông tin dùng để chuyển máy từ trạng thái “ngủ” sang trạng thái hoạt động. Do giá thành đang giảm đi nhanh trong khi dung lượng ngày càng lớn nên bộ nhớ SSD được sử dụng ngày càng phổ biến.

Vào/ra dữ liệu (I/O). Máy tính trao đổi dữ liệu với bên ngoài nhờ một số thiết bị vào/ra dữ liệu hay thiết bị ngoại vi như bàn phím, màn hình, máy in, đĩa .v.v. Vào/ra dữ liệu là quá trình trao đổi dữ liệu giữa CPU và bộ nhớ chính với các thiết bị vào/ra dữ liệu hoặc bộ nhớ ngoài. Mỗi dạng thiết bị vào ra được điều khiển bởi bộ điều khiển (device controller) tương ứng, ví dụ bộ điều khiển SCSI (small computer system interface) thường được dùng để kết nối và điều khiển đĩa cứng. Các bộ điều khiển này được kết nối với CPU qua bus hệ thống. Bộ điều khiển thiết bị có bộ nhớ riêng của mình và các thanh ghi, mỗi thanh ghi được đánh số (địa chỉ), các số này được gọi là cổng vào/ra dữ liệu.

Quá trình vào/ra dữ liệu được thực hiện như sau. CPU ghi một số thông tin vào thanh ghi tương ứng của bộ điều khiển thiết bị cần vào/ra dữ liệu. Nội dung thông tin này là các chỉ thị cho bộ điều khiển thiết bị biết phải làm gì cùng với dữ liệu nếu đó là lệnh ghi ra, ví dụ lệnh ghi ra máy in. Bộ điều khiển sẽ chuyển dữ liệu giữa thanh ghi và thiết bị ngoại vi. Khi quá trình chuyển dữ liệu kết thúc, bộ điều khiển sinh ra ngắt để thông báo cho hệ thống cùng với dữ liệu đọc được (là lệnh đọc dữ liệu). Phương pháp vào ra dữ liệu như vậy gọi là *phương pháp sử dụng ngắt*.

Quy trình vào/ra dữ liệu nêu trên đòi hỏi sự tham gia của CPU và do vậy không hiệu quả khi vào/ra lượng dữ liệu lớn như khi đọc từ đĩa cứng vào bộ nhớ hoặc ngược lại. Trong trường hợp này có thể sử dụng cơ chế vào ra khác là *truy cập trực tiếp bộ nhớ* (DMA – Direct Memory Access). Cơ chế này cho phép truyền lượng dữ liệu lớn giữa đĩa và bộ nhớ chính. CPU chỉ tham gia để xác lập thông tin ban đầu bằng cách ghi địa chỉ vùng nhớ, vùng đệm, số đếm, sau đó việc trao đổi sẽ diễn ra không cần CPU. Khi toàn bộ dữ liệu đã được truyền, bộ điều khiển sẽ sinh ngắt để thông báo. Như vậy, DMA hiệu quả hơn khi trao đổi lượng dữ liệu lớn với đĩa do không đòi hỏi sự tham gia của CPU.

1.3. KHÁI NIỆM HỆ ĐIỀU HÀNH

Có nhiều cách định nghĩa khác nhau về hệ điều hành, nhưng thông thường, hệ điều hành được định nghĩa thông qua mục đích, vai trò, và chức năng trong hệ thống máy tính.

Hệ điều hành là hệ thống phần mềm đóng vai trò trung gian giữa người sử dụng và phần cứng máy tính nhằm tạo ra môi trường giúp thực hiện các chương trình một cách thuận tiện. Ngoài ra, hệ điều hành còn quản lý và đảm bảo cho việc sử dụng phần cứng của máy tính được hiệu quả.

Để hoàn thành vai trò của mình, hệ điều hành cần thực hiện hai chức năng cơ bản là *quản lý tài nguyên* và *quản lý việc thực hiện các chương trình*. Ta sẽ xem xét kỹ hai chức năng này của hệ điều hành.

Quản lý tài nguyên

Quản lý tài nguyên đảm bảo cho tài nguyên hệ thống được sử dụng một cách có ích và hiệu quả. Nhờ có hệ điều hành, tài nguyên được quản lý và sử dụng hợp lý hơn trong khi người sử dụng được giải phóng khỏi công việc khó khăn này.

Các tài nguyên phần cứng chủ yếu của máy tính gồm có bộ xử lý (CPU), bộ nhớ chính, bộ nhớ thứ cấp, các thiết bị vào ra. CPU là thành phần trung tâm của hệ thống, có chức năng xử lý dữ liệu và điều khiển toàn hệ thống. Bộ nhớ chính là nơi lưu trữ chương trình và dữ liệu trong quá trình xử lý. Bộ nhớ thứ cấp, hay *bộ nhớ ngoài*, bao gồm các đĩa từ, đĩa quang học, đĩa quang từ, băng từ, thẻ nhớ và các thiết bị nhớ khác có vai trò lưu trữ chương trình, dữ liệu trong thời gian dài với dung lượng lớn. Thiết bị vào ra cho phép máy tính trao đổi thông tin với thế giới bên ngoài.

Quản lý tài nguyên trước hết là phân phối tài nguyên tới các ứng dụng một cách hiệu quả. Để thực hiện được, các chương trình cần tài nguyên phần cứng như không gian bộ nhớ, thiết bị ngoại vi. Yêu cầu tài nguyên được hệ điều hành thu nhận và đáp ứng bằng cách cấp cho chương trình các tài nguyên tương ứng. Muốn cấp phát tài nguyên, hệ điều hành cần lưu trữ tình trạng tài nguyên để biết hiện giờ tài nguyên nào còn trống, tài nguyên nào đang được sử dụng. Một ví dụ điển hình là trường hợp lưu trữ thông tin lên đĩa. Hệ điều hành cần biết những vùng nào trên đĩa chưa được sử dụng để ghi thông tin lên những vùng này. Việc ghi thông tin lên vùng trống cũng cần được tính toán sao cho quá trình truy cập tới thông tin khi cần có thể thực hiện nhanh nhất.

Yêu cầu về phần cứng của các chương trình này có thể mâu thuẫn nhau. Chẳng hạn, hai chương trình cùng có yêu cầu ghi ra đĩa một lúc. Trong trường hợp xuất hiện các yêu cầu mâu thuẫn khác về phần cứng như ví dụ này, hệ điều hành sẽ quyết định thứ tự và thời gian cung cấp tài nguyên cho các chương trình sao cho đạt được mục tiêu tính toán của hệ thống đồng thời tối ưu hoá một số tiêu chí nào đó, chẳng hạn giảm thời gian các chương trình phải tạm ngừng để chờ đợi lẫn nhau.v.v.

Quản lý tài nguyên còn có nghĩa là đảm bảo sao cho chương trình không xâm phạm tài nguyên đã cấp cho chương trình khác. Ví dụ, nếu hai chương trình được cấp hai vùng bộ nhớ khác nhau, thì việc chương trình này truy cập và thay đổi vùng bộ nhớ của chương trình khác sẽ làm cho chương trình đó hoạt động không bình thường. Hệ điều hành cần thể hiện chức năng quản lý tài nguyên của mình qua việc ngăn ngừa những vi phạm kiểu này.

Quản lý việc thực hiện các chương trình

Nhiệm vụ quan trọng nhất của máy tính là thực hiện các chương trình. Một chương trình đang trong quá trình thực hiện được gọi là tiến trình (process). Chương trình cần được quản lý để có thể thực hiện thuận lợi, tránh các lỗi, đồng thời đảm bảo môi trường để việc xây dựng và thực hiện chương trình được thuận lợi.

Giới thiệu chung

Hệ điều hành giúp việc chạy chương trình dễ dàng hơn. Để chạy chương trình cần thực hiện một số thao tác nhất định, nhờ có hệ điều hành, người dùng không phải thực hiện các thao tác này. Hệ điều hành cũng cung cấp giao diện giúp người dùng dễ dàng chạy hoặc kết thúc các chương trình.

Để tạo môi trường thuận lợi cho chương trình, hệ điều hành tạo ra các máy ảo. Máy ảo là các máy logic với những tài nguyên ảo có các tính chất và khả năng khác so với tài nguyên thực: dễ sử dụng hơn, dễ lập trình hơn, số lượng nhiều hơn tài nguyên thực, khả năng có thể vượt quá khả năng tài nguyên thực.

Tài nguyên ảo là bản mô phỏng của tài nguyên thực được thực hiện bằng phần mềm.

Tài nguyên ảo giống tài nguyên thực ở chỗ nó cung cấp các dịch vụ cơ bản như tài nguyên thực. Chẳng hạn, processor ảo cung cấp khả năng thực hiện các lệnh, bộ nhớ ảo cung cấp khả năng lưu trữ thông tin, thiết bị vào/ra ảo cho phép chương trình đọc ghi dữ liệu.

Tài nguyên ảo khác tài nguyên thực ở chỗ dễ sử dụng hơn. Các tài nguyên thực đều rất khó lập trình trực tiếp. Lấy ví dụ việc ghi thông tin ra đĩa cứng. Các đĩa cứng thường được lập trình bằng cách ghi một số lệnh ra các thanh ghi điều khiển. Các thanh ghi khác làm nhiệm vụ chứa thông tin cần trao đổi và trạng thái đĩa. Để thực hiện việc đọc ghi thông tin, ta cần xác định chuỗi lệnh khởi động (làm đĩa quay nếu đĩa đang ở trạng thái dừng), kiểm tra xem đĩa đã đạt được tốc độ chưa, sau đó chuyển đầu đọc tới vị trí cần thiết, ghi thông tin ra các thanh ghi dữ liệu và đưa các lệnh tiến hành ghi thông tin ra các thanh ghi điều khiển. Việc lập trình điều khiển đĩa như vậy đòi hỏi rất nhiều thời gian của những hiểu biết về giao diện phần cứng. Trong trường hợp này là kiến thức về các lệnh, địa chỉ, khuôn dạng thanh ghi và quá trình trao đổi tin với đĩa. Nếu mạch điều khiển đĩa thay đổi thì các thông số này có thể thay đổi theo và chương trình ghi đĩa cũng phải viết lại.

Để cho việc sử dụng các tài nguyên phần cứng trở nên đơn giản người ta trừu tượng hoá các tài nguyên này. Trừu tượng hoá là quá trình loại bỏ các chi tiết không quan trọng, chỉ giữ lại những khía cạnh cốt lõi mà người sử dụng quan tâm. Các tài nguyên phần cứng sau khi được trừu tượng hoá vẫn cung cấp các chức năng cơ bản như ban đầu xong dễ sử dụng hơn nhiều do các chi tiết cụ thể đã được giấu đi. Chẳng hạn, đĩa cứng có thể coi như nơi có thể đọc, ghi các tệp. Người dùng có thể tạo, xoá, đọc, ghi các tệp bằng các lệnh bậc cao mà không cần quan tâm tới các thanh ghi, các lệnh bậc thấp. Việc trực tiếp đưa các lệnh cụ thể ra thanh ghi cùng các chi tiết khác sẽ do hệ điều hành đảm nhiệm.

Một điểm khác biệt quan trọng của tài nguyên ảo là số lượng tài nguyên ảo có thể lớn hơn số lượng tài nguyên thực. Hãy xem xét trường hợp CPU. Mỗi máy tính thường chỉ có một processor thực. Tuy nhiên nếu nhiều chương trình cùng được thực hiện trên máy đó, mỗi chương trình sẽ được hệ điều hành cung cấp một CPU ảo bằng cách phân chia thời gian sử dụng CPU thực cho các CPU ảo đó. Rõ ràng số lượng processor ảo lúc đó vượt số lượng CPU thực rất nhiều. Khả năng của từng tài nguyên ảo cũng có thể vượt khả năng tài nguyên thực. Điển hình là bộ nhớ ảo. Các hệ điều hành thường cung cấp bộ nhớ trong ảo với không gian nhớ lớn hơn bộ nhớ thực rất nhiều bằng cách sử dụng thêm không gian trên bộ nhớ ngoài.

1.4. CÁC DỊCH VỤ DO HỆ ĐIỀU HÀNH CUNG CẤP

Một trong các nhiệm vụ chủ yếu của hệ điều hành là tạo ra môi trường thuận lợi cho các chương trình khác thực hiện và giúp người dùng sử dụng hệ thống dễ dàng. Điều này thể hiện qua một số dịch vụ mà hệ điều hành cung cấp cho các chương trình ứng dụng và người sử dụng. *Khái niệm dịch vụ ở đây có thể hiểu đơn giản là những công việc mà hệ điều hành thực hiện giúp người dùng hoặc chương trình ứng dụng.*

Các dịch vụ có thể thay đổi theo từng hệ điều hành. Một số hệ điều hành cung cấp nhiều dịch vụ trong khi hệ điều hành khác cung cấp ít dịch vụ hơn. Chẳng hạn, MS-DOS không cung cấp các dịch vụ về bảo mật trong khi Windows NT và các phiên bản sau lại rất chú trọng tới dịch vụ này. Tuy nhiên có một số dịch vụ mà một hệ điều hành tiêu biểu thường có.

Dưới đây là những dịch vụ thường gặp của hệ điều hành.

- **Tải và chạy chương trình.** Để thực hiện một chương trình, chương trình đó cần được tải từ đĩa vào bộ nhớ, sau đó được trao quyền thực hiện các lệnh. Khi chương trình đã thực hiện xong cần giải phóng bộ nhớ và các tài nguyên mà chương trình chiếm giữ. Toàn bộ quá trình này tương đối phức tạp song lại diễn ra thường xuyên. Hệ điều hành sẽ thực hiện công việc phức tạp và lặp đi lặp lại này. Nhờ có hệ điều hành, lập trình viên cũng như người sử dụng không cần quan tâm tới chi tiết của việc tải và chạy chương trình.
- **Giao diện với người dùng.** Các hệ thống thường cung cấp giao diện cho phép hệ điều hành giao tiếp với hệ điều hành. Hai dạng giao diện thông dụng nhất là giao diện dưới dạng dòng lệnh (command-line) và giao diện đồ họa (Graphic User Interface – GUI). Giao diện dòng lệnh cho phép người dùng ra chỉ thị cho hệ điều hành bằng cách gõ lệnh dưới dạng văn bản, ví dụ chương trình cmd.exe của Windows. Giao diện đồ họa sử dụng hệ thống cửa sổ thực đơn, và thiết bị trỏ như chuột, kết hợp với bàn phím để giao tiếp với hệ thống.
- **Thực hiện các thao tác vào ra dữ liệu.** Người dùng và chương trình trong khi thực hiện có thể có nhu cầu vào/ra dữ liệu với đĩa hoặc các thiết bị ngoại vi. Để thực hiện vào/ra, cần ghi các lệnh được xác định sẵn ra những thanh ghi nhất định của bộ điều khiển thiết bị ngoại vi, gọi là cổng vào/ra, sau đó đọc lại kết quả hoặc trạng thái của thao tác vào/ra. Để tránh cho chương trình không phải làm việc trực tiếp với phần cứng qua nhiều bước phức tạp như vậy, yêu cầu vào/ra sẽ được giao cho hệ điều hành thực hiện.
- **Làm việc với hệ thống file.** File là một khái niệm lô gic dùng để trừu tượng hoá công việc vào ra thông tin với bộ nhớ ngoài. Đa số người dùng và chương trình có nhu cầu đọc, ghi, tạo, xóa, chép file hoặc làm việc với thư mục. Ngoài ra còn nhiều thao tác khác với file như quản lý quyền truy cập, sao lưu. Hệ điều hành giúp thực hiện những thao tác này dưới dạng các dịch vụ.
- **Phát hiện và xử lý lỗi.** Để đảm bảo cho hệ thống hoạt động ổn định, an toàn, hệ điều hành cần phát hiện và xử lý kịp thời các lỗi xuất hiện trong phần cứng cũng như phần mềm. Các lỗi phần cứng có thể là lỗi bộ nhớ, mất điện, máy in hết giấy.v.v. Các lỗi

Giới thiệu chung

phần mềm có thể do chương trình viết sai, các phép chia cho không, lỗi truy cập bộ nhớ.v.v. Nếu không có hệ điều hành, người dùng và chương trình ứng dụng sẽ phải tự phát hiện và xử lý các lỗi xuất hiện.

- **Truyền thông.** Trong khi thực hiện, chương trình có thể có nhu cầu trao đổi thông tin với nhau, thậm chí với chương trình đang thực hiện trên máy khác được nối mạng. Hệ điều hành cung cấp dịch vụ cho phép thiết lập liên lạc và truyền thông tin dưới dạng các thông điệp (message) hoặc thông qua những vùng bộ nhớ dùng chung (shared memory). Trong trường hợp truyền thông điệp, hệ điều hành đóng vai trò chuyển các gói tin theo những quy tắc nhất định gọi là giao thức truyền thông.
- **Cấp phát tài nguyên.** Trong các hệ thống cho phép nhiều chương trình thực hiện đồng thời cần có cơ chế cấp phát và phân phối tài nguyên hợp lý. Mỗi dạng tài nguyên cần có cách cấp phát riêng, ví dụ cơ chế cấp phát CPU hoàn toàn khác so với cấp phát bộ nhớ. Nhờ có hệ điều hành, người sử dụng và trình ứng dụng không phải tự thực hiện việc cấp phát tài nguyên, đồng thời vẫn đảm bảo tài nguyên được cấp phát công bằng và hiệu quả.
- **Dịch vụ an ninh và bảo mật.** Đối với hệ thống nhiều người dùng thường có xuất hiện yêu cầu bảo mật thông tin, tức là đảm bảo người dùng này không tiếp cận được với thông tin của người khác nếu không được phép. Tương tự như vậy, hệ thống cần đảm bảo để tiến trình không truy cập trái phép tài nguyên (như vùng bộ nhớ, file mở) của tiến trình khác hay của chính hệ điều hành bằng cách kiểm soát truy cập tới tài nguyên. Nhiều hệ điều hành còn cho phép kiểm tra người dùng thông qua việc kiểm soát đăng nhập vào hệ thống.

Tải và chạy hệ điều hành

Ở đây cần nói thêm về việc tải hệ điều hành vào bộ nhớ. Như đã nói ở trên, việc tải các chương trình vào bộ nhớ là do hệ điều hành thực hiện. Do hệ điều hành là chương trình đầu tiên được thực hiện khi khởi động hệ thống nên hệ điều hành phải tự tải chính mình từ bộ nhớ ngoài vào bộ nhớ trong. Chính xác hơn, quá trình đó, được gọi là booting (viết tắt của bootstrapping), diễn ra như sau. Hệ điều hành có một chương trình nhỏ gọi là chương trình tải hay chương trình môi (OS loader hoặc boot). Chương trình này nằm ở một vị trí xác định trên đĩa hoặc thiết bị nhớ ngoài khác. Chẳng hạn, đối với đĩa, chương trình này nằm ở sector đầu tiên của đĩa. Việc chương trình tải nằm ở vị trí xác định như vậy là rất quan trọng vì nếu không, phần cứng sẽ không thể tìm ra chương trình tải hệ điều hành.

Sau khi khởi động hệ thống, một chương trình nằm sẵn trong bộ nhớ ROM (ví dụ trong BIOS của máy vi tính) sẽ được kích hoạt và đọc chương trình môi của hệ điều hành từ vị trí quy ước trên đĩa vào bộ nhớ. Sau đó, chương trình môi chịu trách nhiệm tải các phần khác của hệ điều hành vào bộ nhớ và trao cho hệ điều hành quyền điều khiển hệ thống. Nếu phần đĩa chứa chương trình môi bị hỏng, phần cứng sẽ hiển thị thông báo với nội dung “không tìm thấy hệ điều hành”.

Trong trường hợp máy tính được cài nhiều hệ điều hành, chương trình môi (gọi là Multi OS loader) sẽ cho phép người dùng chọn một trong các hệ điều hành đó để tải vào bộ nhớ.

1.5. GIAO DIỆN LẬP TRÌNH CỦA HỆ ĐIỀU HÀNH

Để các chương trình có thể sử dụng được những dịch vụ nói trên, hệ điều hành cung cấp một giao diện gọi là *giao diện lập trình*. Giao diện này bao gồm các *lời gọi hệ thống* (system calls) mà chương trình sử dụng để yêu cầu một dịch vụ nào đó từ phía hệ điều hành.

Lời gọi hệ thống là dạng lệnh đặc biệt mà chương trình ứng dụng gọi khi cần yêu cầu hệ điều hành thực hiện một việc gì đó. Các hệ điều hành trước đây thường cung cấp lời gọi hệ thống dưới dạng các lệnh hợp ngữ do đó lời gọi hệ thống còn được gọi là “lệnh máy mở rộng”. Ví dụ các lời gọi kiểu này là các hàm ngắt 21h của DOS mà chương trình viết trên hợp ngữ gọi bằng lệnh int. Hệ điều hành hiện nay thường cho phép gọi lời gọi hệ thống trực tiếp từ ngôn ngữ bậc cao như C hoặc C++. Lúc này, lời gọi hệ thống giống như một lời gọi hàm hoặc chương trình con thông thường. Trên hình 1.4 là ví dụ một lời gọi hệ thống của hệ điều hành Windows cho phép ghi ra file.

Trên thực tế, chương trình ứng dụng ít sử dụng trực tiếp lời gọi hệ thống. Thay vào đó, lời gọi hệ thống được thực hiện qua những thư viện hàm gọi là thư viện hệ thống cùng với những hàm hệ thống khác. Các hàm này sẽ giúp người lập trình gọi lời gọi hệ thống tương ứng của hệ điều hành. Giao diện lập trình Win32 API (Application Programming Interface) do hệ điều hành Windows cung cấp là một ví dụ về thư viện như vậy. Các ví dụ khác là POSIX API dùng cho UNIX, Linux và Java API dùng cho máy ảo Java.

```
NTSTATUS ZwWriteFile(
    _In_      HANDLE FileHandle,
    _In_opt_  HANDLE Event,
    _In_opt_  PIO_APC_ROUTINE pApcRoutine,
    _In_opt_  PVOID ApcContext,
    _Out_     PIO_STATUS_BLOCK IoStatusBlock,
    _In_      PVOID Buffer,
    _In_      ULONG Length,
    _In_opt_  PLARGE_INTEGER ByteOffset,
    _In_opt_  PULONG Key
);
```

Hình 1.4. Lời gọi hệ thống với dịch vụ ghi ra file của Windows

Trên hình 1.5 là ví dụ một hàm của Win32 API cho phép yêu cầu dịch vụ ghi ra file của Windows. Có thể so sánh hàm này với hàm zwWriteFile trong ví dụ ở hình trên để thấy mối quan hệ giữa lời gọi hệ thống và API.

```
BOOL WINAPI WriteFile(
    __in      HANDLE hFile,
    __in      LPCVOID lpBuffer,
    __in      DWORD nNumberOfBytesToWrite,
    __out_opt LPDWORD lpNumberOfBytesWritten,
    __inout_opt LPOVERLAPPED lpOverlapped
);
```

Hình 1.5 : Hàm ghi file trong thư viện Windows API

Giới thiệu chung

Khi viết chương trình, người lập trình sẽ sử dụng các hàm do giao diện lập trình ứng dụng API cung cấp thay vì gọi trực tiếp lời gọi hệ thống. Chương trình dịch (compiler) sau đó sẽ thực hiện việc chuyển đổi lời gọi hàm sang lời gọi hệ thống tương ứng của hệ điều hành. Trên thực tế, đa số API và lời gọi hệ thống có hình thức khá tương tự nhau như trong ví dụ ở hình trên.

Việc sử dụng API có một số ưu điểm so với sử dụng trực tiếp lời gọi hệ thống.

- Thứ nhất, chương trình dễ dàng chuyển sang thực hiện trên hệ thống khác có cùng API. Khi hệ điều hành nâng cấp lời gọi hệ thống bằng cách thêm chức năng mới, chương trình gọi trực tiếp lời gọi hệ thống sẽ phải viết lại, trong khi chương trình sử dụng API thì không.
- Thứ hai, hàm API thường thực hiện thêm một số thao tác so với lời gọi hệ thống tương ứng. Ví dụ, khi thực hiện lời gọi hệ thống để tạo tiến trình mới, cần làm một số thao tác trước và sau khi lời gọi này. Hàm API tương ứng đã chứa sẵn đoạn mã thực hiện các thao tác này, do vậy chương trình gọi hàm API sẽ không phải tự thực hiện các thao tác.
- Thứ ba, hàm API thường hỗ trợ các phiên bản khác nhau của hệ điều hành và tự phát hiện phiên bản phù hợp.

Các lời gọi hệ thống và hàm API thường thuộc vào trong các nhóm sau: quản lý tiến trình, quản lý file và thư mục, quản lý thiết bị, đảm bảo thông tin và liên lạc giữa các tiến trình. Trên hình 1.6 là ví dụ một số hàm API quan trọng trong thư viện Win32 API của Windows và POSIX cho Unix và Linux. Lưu ý rằng đây hình này chỉ liệt kê một số lượng nhỏ hàm API trong số hàm trăm hàng của hệ thống.

POSIX	Win32	Mô tả
Fork	CreateProcess	Tạo tiến trình mới
execve	—	CreateProcess = fork + execve
Exit	ExitProcess	Kết thúc tiến trình
Open	CreateFile	Tạo mới file hoặc mở một file đã có
Close	CloseHandle	Đóng một file đang mở
Read	ReadFile	Đọc từ file
Write	WriteFile	Ghi ra file
Lseek	SetFilePointer	Di chuyển tới một vị trí cụ thể trong file
Stat	GetFileAttributeExt	Đọc các thuộc tính của file
Mkdir	CreateDirectory	Tạo thư mục mới
Rmdir	RemoveDirectory	Xóa thư mục
Unlink	DeleteFile	Xóa một file

Chdir	SetCurrentDirectory	Thay đổi thư mục hiện thời
-------	---------------------	----------------------------

Hình 1.6. Ví dụ một số hàm API quan trọng của POSIX và Win32 API

1.6. QUÁ TRÌNH PHÁT TRIỂN VÀ MỘT SỐ KHÁI NIỆM QUAN TRỌNG

Các hệ điều hành ngày nay là những hệ thống phần mềm phức tạp thực hiện nhiều chức năng tinh vi liên quan tới quản lý tài nguyên và chương trình. Các tính năng và kỹ thuật được sử dụng trong hệ điều hành hiện đại không phải có ngay mà được xây dựng và hoàn thiện qua nhiều thế hệ hệ điều hành khác nhau. Do vậy, việc xem xét quá trình phát triển hệ điều hành cho phép hiểu rõ hơn khả năng và yêu cầu đối với một hệ điều hành hiện đại.

Trong phần này cũng trình bày một số khái niệm quan trọng như đa chương trình, đa nhiệm, chia sẻ thời gian.

Các hệ thống đơn giản

Trong thời kỳ mới ra đời, từ giữa những năm 40 cho tới giữa những năm 50 thế kỷ trước, tốc độ xử lý của máy tính rất thấp, việc vào/ra được thực hiện thủ công và khó khăn. Việc nạp chương trình được thực hiện nhờ các công tắc, các mạch hàn sẵn (plugboard), bìa đục lỗ. Kết quả thực hiện được đưa ra máy in, trạng thái máy thể hiện trên các đèn tín hiệu. Trong thời kỳ này, lập trình viên tương tác trực tiếp với phần cứng, lập trình bằng các lệnh máy. Máy tính điện tử hệ này *chưa có hệ điều hành*.

Xử lý theo mẻ

Từ giữa những năm 1950, phần cứng máy tính đã có những cải tiến quan trọng. Việc sử dụng bán dẫn cho phép giảm kích thước máy, tăng tốc độ xử lý cũng như giảm các hỏng hóc phần cứng. Việc nạp chương trình được thực hiện nhờ bìa đục lỗ vào các đĩa từ trước khi tải vào máy. Hệ điều hành đầu tiên cũng ra đời trong thời kỳ này.

Trong những thập niên đầu tiên khi ra đời, giá thành máy tính rất đắt. Do đó, nhiệm vụ quan trọng là tận dụng hết công suất máy, giảm thời gian chờ đợi càng nhiều càng tốt. Một kỹ thuật cho phép tăng hiệu suất sử dụng máy là *xử lý theo mẻ* (batch processing), hay còn gọi là *xử lý theo lô*. Kỹ thuật này lần đầu tiên được hãng General Motors sử dụng trên máy tính 701 vào giữa những năm 1950.

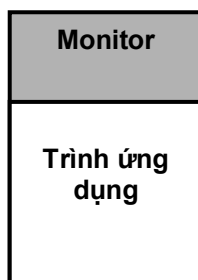
Thay vì làm việc trực tiếp với máy tính, lập trình viên chuẩn bị chương trình trên bìa đục lỗ hoặc trên đĩa từ và giao cho các kỹ thuật viên. Đây là những người chuyên trách quản lý máy và được chuẩn bị để sử dụng máy hiệu quả nhất. Sau khi nhận được chương trình, kỹ thuật viên sẽ phân chương trình thành các *mẻ*. Mỗi mẻ bao gồm những chương trình có yêu cầu giống nhau, ví dụ các chương trình cần được dịch bằng bộ dịch FORTRAN được xếp vào cùng mẻ. Toàn bộ mẻ sau đó được nạp vào băng từ và được tải vào máy để thực hiện lần lượt.

Để có thể tự động hóa xử lý theo mẻ, một chương trình nhỏ gọi là *chương trình giám sát* (monitor) được giữ thường xuyên trong bộ nhớ. Mỗi khi một chương trình của mẻ kết thúc, chương trình giám sát tự động nạp chương trình tiếp theo của mẻ vào máy và cho phép chương trình này chạy. Việc tự động hoá giám sát và nạp chương trình còn giảm đáng kể thời gian chuyển đổi giữa hai chương trình trong cùng một mẻ do monitor có thể tự động nạp

Giới thiệu chung

chương trình nhanh hơn kỹ thuật viên. Hiệu suất sử dụng CPU do đó được cải thiện đáng kể. Sau khi toàn bộ mẽ đã được thực hiện xong, kỹ thuật viên lấy băng từ chứa mẽ ra và nạp tiếp mẽ mới vào để thực hiện.

Trình giám sát (monitor) mô tả ở trên chính là *dạng đơn giản nhất của hệ điều hành* được tải vào và nằm thường trực trong bộ nhớ để quản lý việc thực hiện các chương trình khác. Bộ nhớ máy tính được phân thành hai vùng: một vùng chứa trình giám sát, và một vùng



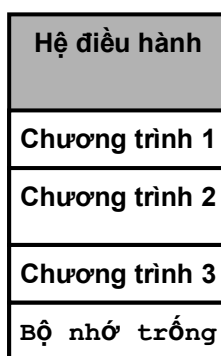
Hình 1.7: Bộ nhớ chứa trình giám sát (monitor) và chương trình ứng dụng

chứa trình ứng dụng như minh họa trên hình 1.7.

Đa chương trình

Mặc dù việc xử lý theo mẽ cho phép giảm thời gian chuyển đổi giữa các chương trình ứng dụng xong hiệu suất sử dụng CPU vẫn tương đối thấp. Mỗi khi có yêu cầu vào/ra, CPU phải dừng việc xử lý dữ liệu để chờ quá trình vào/ra kết thúc. Do tốc độ vào ra luôn thấp hơn tốc độ CPU rất nhiều nên CPU thường xuyên phải chờ đợi trong những khoảng thời gian dài.

Để hạn chế tình trạng nói trên, kỹ thuật *đa chương trình* (multiprogramming) được sử dụng. Hệ thống chứa đồng thời nhiều chương trình trong bộ nhớ (hình 1.8). Khi một chương trình phải dừng lại để thực hiện vào ra hệ điều hành sẽ chuyển CPU sang thực hiện một chương trình khác. Nếu số chương trình nằm trong bộ nhớ đủ nhiều thì hầu như lúc nào CPU cũng có việc để thực hiện, nhờ vậy giảm thời gian chạy không tải của CPU.

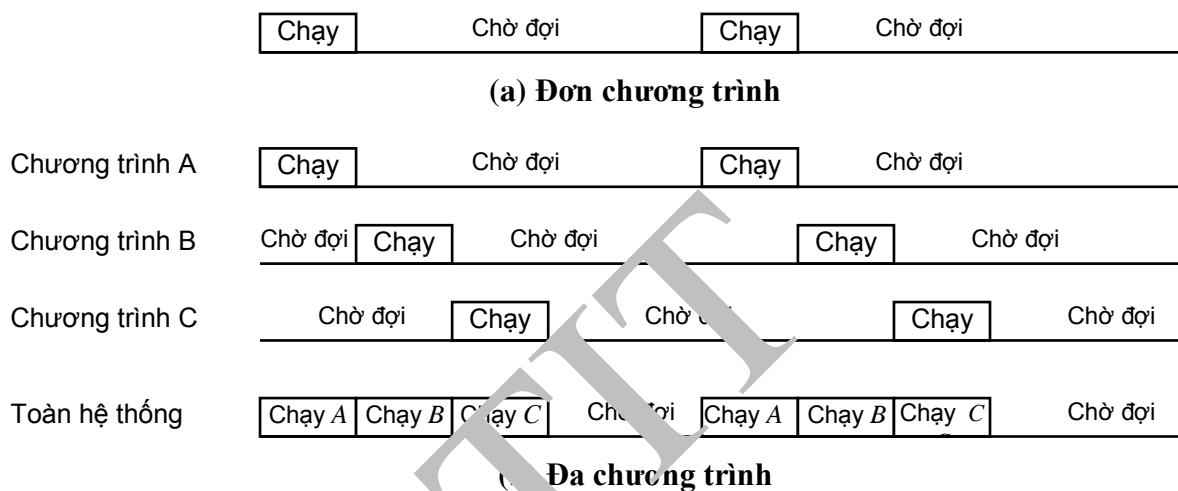


Hình 1.8: Đa chương trình

Trên hình 1.9 là minh họa hiệu suất sử dụng thời gian CPU cho trường hợp đơn chương trình và đa chương trình với 3 chương trình cùng được tải vào bộ nhớ một lúc. Thời gian thực

hiện chương trình xen kẽ với thời gian chờ đợi vào/ra. Dễ dàng nhận thấy, thời gian chờ đợi của CPU trong chế độ đa chương trình giảm đáng kể so với trong trường hợp đơn chương trình.

Trong trường hợp đa chương trình, hệ điều hành trở nên phức tạp hơn rất nhiều so với trường hợp đơn chương trình. Trước hết, cần quyết định xem bao nhiêu chương trình được tải vào bộ nhớ. Sau khi đã các chương trình đã ở trong bộ nhớ và sẵn sàng thực hiện (gọi là các tiến trình), hệ điều hành phải phân phối CPU cho các tiến trình. Việc phân phối CPU như vậy gọi là *điều độ tiến trình* hay *điều độ CPU* và sẽ được trình bày chi tiết trong chương 2. Ngoài ra, hệ điều hành cần đảm bảo để tiến trình không xâm phạm vùng nhớ và tài nguyên đã cấp cho tiến trình khác.



Hình 1.9: Chế độ đơn chương trình và đa chương trình

Việc thực hiện đa chương trình đòi hỏi những sự hỗ trợ nhất định từ phần cứng, đặc biệt là khả năng vào/ra bằng ngắt và cơ chế DMA. Nếu không có cơ chế này, CPU sẽ phải trực tiếp điều khiển quá trình vào/ra thông tin và dữ liệu. Hiệu quả của đa chương trình do đó sẽ bằng không.

Chia sẻ thời gian và đa nhiệm

Mặc dù đa chương trình cho phép sử dụng hiệu quả CPU và các tài nguyên khác của hệ thống, song kỹ thuật này không cho phép người dùng tương tác với hệ thống. Trong các máy tính thế hệ sau, các terminal cho phép người dùng làm việc trực tiếp với máy tính thông qua màn hình và bàn phím. Nhiều người dùng có thể nhập thông tin và lệnh từ bàn phím, kết quả sau đó được đưa trực tiếp ra màn hình. Đối với các hệ thống này, thời gian đáp ứng, tức là thời gian từ khi người dùng gõ lệnh cho tới khi máy tính phản xạ lại cần phải tương đối nhỏ. Kỹ thuật đa chương trình mô tả ở trên không đảm bảo được thời gian đáp ứng ngắn như vậy. Do vậy, một kỹ thuật khác gọi là *chia sẻ thời gian* được sử dụng. Các hệ thống đa chương trình có chia sẻ thời gian được gọi là hệ thống *đa nhiệm* (multitasking), theo đó các chương trình được gọi là các *nhiệm vụ* (task).

Chia sẻ thời gian có thể coi như đa chương trình cải tiến. CPU lần lượt thực hiện các

Giới thiệu chung

công việc khác nhau trong những khoảng thời gian ngắn gọi là lượng tử thời gian. Do việc chuyển đổi giữa các công việc diễn ra với tần số cao và tốc độ CPU lớn nên thời gian đáp ứng nằm trong giới hạn có thể chấp nhận, tất cả người dùng đều có cảm giác máy tính chỉ thực hiện chương trình của mình.

Như vậy, trong chế độ chia sẻ thời gian, CPU được chia sẻ giữa những người dùng khác nhau tương tác trực tiếp với hệ thống. Hệ điều hành sử dụng các kỹ thuật đa chương trình và điều độ CPU để cung cấp CPU cho người dùng trong những khoảng thời gian ngắn. Mỗi người dùng sẽ có chương trình của mình (một hoặc nhiều) trong bộ nhớ. Các chương trình đang thực hiện như vậy được gọi là tiến trình. Hệ điều hành chuyển quyền sử dụng CPU giữa các tiến trình khác nhau.

Hệ điều hành hỗ trợ chia sẻ thời gian phức tạp hơn hệ điều hành đa chương trình đơn thuần rất nhiều. Để đảm bảo chia sẻ CPU, hệ điều hành phải có các cơ chế điều độ tiến trình phức tạp, cho phép đồng bộ hoá, đảm bảo liên lạc giữa các tiến trình, cũng như tránh tình trạng bế tắc.

Tính toán di động

Một xu hướng mới của các hệ thống máy tính là tính toán di động, tức là môi trường tính toán trên các thiết bị cầm tay như điện thoại di động thông minh (smart phone), máy tính bảng (tablet computer), hay thiết bị trợ giúp cá nhân. Do sự phát triển của phần cứng, thiết bị di động dù có kích thước nhỏ nhưng có khả năng tính toán và lưu trữ thông tin khá mạnh, ngày càng gần với khả năng của máy tính thông thường. Ngoài các chức năng truyền thống như truyền thông, thư điện tử, duyệt web, thiết bị di động dần cung cấp thêm nhiều chức năng chụp ảnh, sách điện tử, chơi nhạc, video, chơi game, các ứng dụng văn phòng đơn giản, các chức năng dựa trên việc xác định vị trí của thiết bị (thông qua hệ thống định vị GPS) .v.v.

Hệ điều hành cho các hệ thống tính toán di động thường chú trọng tới một số điểm khác biệt so với hệ điều hành thông thường. Trước hết, giao diện với người dùng cần thân thiện, giúp người dùng dễ dàng tương tác với thiết bị thông qua thiết bị vào ra thông tin hạn chế hơn so với PC. Ví dụ điển hình là việc hỗ trợ rộng rãi màn hình cảm ứng đa điểm và bàn phím ảo. Một đặc điểm khác của hệ điều hành cho thiết bị dạng này là khả năng hỗ trợ các giao thức truyền thông như các giao thức không dây (wifi) hay giao thức truyền thông trong các mạng tế bào (cellular network) các thế hệ 2G, 3G, 4G, cũng như các ứng dụng phục vụ truyền thông, điển hình là thư điện tử, trình duyệt web, tin nhắn, thoại. Hệ điều hành cũng được thiết kế để tối ưu việc sử dụng năng lượng, tăng thời gian sử dụng pin.

Các hệ điều hành phổ biến nhất cho tính toán di động hiện nay là Androi của Google, iOS của hãng Apple, và mới đây có thêm Windows phone của Microsoft.

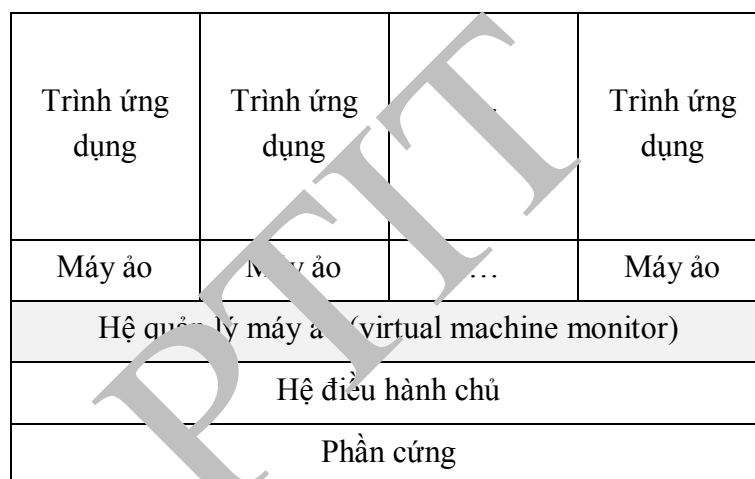
Ảo hóa.

Ảo hóa (virtualization) là kỹ thuật cho phép chạy một hệ điều hành như một ứng dụng của một hệ điều hành khác, thay vì phải chạy trực tiếp trên phần cứng như thông thường. Ví dụ, trên một máy tính đang chạy hệ điều hành Linux ta có thể chạy một máy ảo với hệ điều hành Windows và thực hiện các ứng dụng viết cho Windows trong máy ảo này. Hệ điều hành chạy trực tiếp trên phần cứng được gọi là *hệ chủ* (host). Một hệ chủ có thể hỗ trợ nhiều *máy*

ảo (virtual machines), mỗi máy ảo tương ứng với một hệ điều hành khác.

Kỹ thuật ảo hóa cho phép chạy đồng thời nhiều hệ điều hành trên cùng một máy tính. Các hệ điều hành có thể khác nhau - như Windows và Linux - hoặc là các bản của cùng một hệ điều hành, ví dụ chạy nhiều bản Windows trên một máy. Nhờ vậy, trong trường hợp có nhiều ứng dụng được viết cho các hệ điều hành khác nhau, thay vì phải viết và dịch lại ứng dụng cho hệ điều hành đang chạy trên máy, ta có thể chạy các máy ảo với hệ điều hành tương ứng với ứng dụng.

Các hệ thống hỗ trợ ảo hóa thường có cấu trúc như trên hình 1.10. Phần quan trọng nhất là *hệ quản lý máy ảo* (virtual machine manager, còn gọi là virtual machine monitor, hay hypervisor). Hệ có thể chạy như một ứng dụng của hệ điều hành chủ và tạo ra các máy ảo (virtual machine), mỗi máy có thể chạy một hệ điều hành riêng rẽ. Hệ quản lý máy ảo chịu trách nhiệm liên lạc giữa các hệ điều hành trên máy ảo với CPU, quản lý việc sử dụng tài nguyên của các hệ điều hành này và ngăn chặn việc xâm nhập trái phép giữa các máy ảo khác nhau. Hiện nay, công nghệ và sản phẩm ảo hóa chủ yếu do hãng VMware và Microsoft cung cấp.



Hình 1.10: Kiến trúc hệ thống hỗ trợ ảo hóa

Điện toán đám mây

Điện toán đám mây (cloud computing) là mô hình tính toán phát triển mạnh trong vài năm gần đây và là một trong những xu hướng phát triển quan trọng của các hệ thống tính toán. Trong điện toán đám mây, việc tính toán, lưu trữ dữ liệu cũng như các chương trình ứng dụng được cung cấp qua mạng, chẳng hạn qua Internet, dưới dạng dịch vụ.

Ví dụ, dịch vụ đám mây của hãng Amazon cho phép người dùng thuê một máy tính với cấu hình do người dùng tự xác định. Thời gian thuê có thể từ một giờ tới hàng năm. Như vậy, thay vì mua một máy tính, người dùng có thể thuê máy tính từ dịch vụ Amazon elastic computing cloud (EC2) của Amazon trong khoảng thời gian cần thiết. Người dùng sau đó có thể chạy các chương trình của mình trên máy tính đã thuê và chỉ trả tiền cho thời gian thuê thay vì phải mua máy vĩnh viễn. Điểm quan trọng là máy tính do EC2 cung cấp thực chất là một máy ảo. Từ các server của mình, chương trình quản lý EC2 của Amazon sẽ sử dụng công nghệ ảo hóa để tạo ra một máy ảo có cấu hình như người dùng yêu cầu vào thời gian mà

Giới thiệu chung

người dùng yêu cầu. Công nghệ này đảm bảo tính mềm dẻo và thuận lợi hơn nhiều so với việc mua hoặc thuê một máy tính thực.

Tương tự như vậy, thay vì mua một phần mềm, chẳng hạn Microsoft Office, người dùng có thể thuê phần mềm này trong một thời gian nhất định và chỉ trả tiền cho thời gian thuê. Office 365 của Microsoft là một ví dụ cho dạng dịch vụ phần mềm này.

Điện toán đám mây bao gồm ba dạng dịch vụ chính sau:

- SaaS: viết tắt của Software as a Service, có thể dịch là “phần mềm như một dịch vụ”: cho thuê/cung cấp phần mềm ứng dụng qua mạng.
- PaaS, viết tắt của Platform as a Service, có thể dịch là “nền tảng như một dịch vụ”: là hình thức cho thuê/cung cấp một tập phần mềm đã cài sẵn qua mạng, ví dụ hệ điều hành mạng + hệ quản trị cơ sở dữ liệu để làm máy chủ dữ liệu.
- IaaS, viết tắt của Infrastructure as a Service, có thể dịch là “hạ tầng như một dịch vụ”: là dịch vụ cho thuê/cung cấp phần cứng (CPU, bộ nhớ ngoài) qua mạng.

Điện toán đám mây dựa trên một loạt công nghệ, trong đó quan trọng nhất là công nghệ ảo hóa. Các hệ thống điện toán đám mây thường được xây dựng dựa trên hệ điều hành truyền thống, sau đó bổ sung các phần mềm thực hiện ảo hóa như hệ quản lý máy ảo (VMM) đã nhắc tới ở trên. Các hệ thống quản lý máy ảo cũng có thể tích hợp với hệ điều hành như trường hợp Windows Azure của Microsoft.

1.7. CẤU TRÚC HỆ ĐIỀU HÀNH

Hệ điều hành là một hệ thống phần mềm phức tạp được tạo thành từ nhiều thành phần đảm đương những nhiệm vụ khác nhau, cung cấp những dịch vụ khác nhau. Các thành phần được tổ chức và liên kết với nhau theo một cách nhất định để tạo ra hệ điều hành hoàn chỉnh. Từng thành phần cũng như cách tổ chức toàn bộ hệ thống có thể rất khác nhau, tùy vào hệ điều hành cụ thể. Cách tổ chức, liên kết các thành phần xác định cấu trúc của hệ điều hành. Trong phần này ta sẽ xem xét các thành phần thường có của một hệ điều hành tiêu biểu, sau đó xem xét một số kiểu cấu trúc thông dụng nhất.

1.7.1. Các thành phần của hệ điều hành

Một hệ điều hành tiêu biểu thường có các thành phần thực hiện những nhiệm vụ sau:

a. Quản lý tiến trình

Một chương trình đang trong quá trình thực hiện được gọi là tiến trình. Điểm khác nhau cơ bản giữa chương trình và tiến trình ở chỗ chương trình là một thực thể tĩnh, có thể dưới dạng những bit, những byte ghi trên đĩa, còn chương trình là một thực thể động đang tiến hành việc tính toán, xử lý.v.v. và được cung cấp một số tài nguyên như thời gian CPU, bộ nhớ.v.v. (khái niệm tiến trình sẽ được xem xét kỹ trong các chương sau). Bản thân các tiến trình của hệ điều hành trong khi chạy cũng tạo ra các tiến trình.

Các công việc liên quan tới quản lý tiến trình bao gồm:

- Tạo và xoá tiến trình (bao gồm cả tiến trình người dùng lẫn tiến trình hệ thống - tiến trình hệ điều hành). Lưu thông tin về các tiến trình.
- Tạm treo và khôi phục các tiến trình bị treo. Một tiến trình bị treo sẽ bị tạm dừng và có thể bị chuyển từ bộ nhớ trong ra đĩa. Khi được khôi phục, tiến trình sẽ thực hiện tiếp từ điểm bị treo thay vì thực hiện lại từ đầu. Người sử dụng Linux có thể treo một tiến trình bằng cách sử dụng lệnh suspend.
- Lập lịch cho các tiến trình (process scheduling), hay còn gọi là lập lịch cho CPU, là quyết định tiến trình nào được cấp phát CPU để chạy
- Đồng bộ hoá các tiến trình: khi có nhiều tiến trình cũng tồn tại cần đảm bảo để các tiến trình được thực hiện sao cho không dẫn tới xung đột về tài nguyên hoặc có thể hợp tác với nhau để dẫn tới kết quả mong muốn.
- Giải quyết các bế tắc, ví dụ như khi có xung đột về tài nguyên.
- Tạo cơ chế liên lạc giữa các tiến trình.

b. Quản lý bộ nhớ

Bộ nhớ (nếu không nói gì thêm thì được hiểu là bộ nhớ trong hay bộ nhớ sơ cấp, hay RAM) là nơi chứa các tiến trình và dữ liệu. Đây là tài nguyên quan trọng thứ hai sau CPU. Bộ nhớ là khối ô nhớ được nhóm lại thành các từ hay các byte và được đánh địa chỉ. Địa chỉ được sử dụng khi cần đọc hoặc ghi thông tin vào bộ nhớ. Trong những hệ điều hành đa nhiệm, nhiều tiến trình có thể cùng thực hiện một lúc và được chứa trong bộ nhớ.

Thành phần quản lý bộ nhớ của hệ điều hành thực hiện các công việc sau:

- Cấp phát, phân phối bộ nhớ cho các tiến trình.
- Tạo ra bộ nhớ ảo và ánh xạ địa chỉ bộ nhớ ảo vào địa chỉ bộ nhớ thực. Ngăn chặn các truy cập bộ nhớ không hợp lệ, chẳng hạn truy cập sang vùng bộ nhớ không thuộc tiến trình.
- Cung cấp và giải phóng bộ nhớ theo yêu cầu của các tiến trình.
- Quản lý không gian nhớ đã được cấp và không gian còn trống.

c. Quản lý vào ra

Một trong các nhiệm vụ của hệ điều hành là đơn giản hoá và tăng hiệu quả quá trình trao đổi thông tin giữa các tiến trình với thiết bị vào ra. Nhờ có hệ điều hành, người dùng không phải quan tâm tới các chi tiết liên quan tới thiết bị vào ra cụ thể. Việc điều khiển trực tiếp thiết bị do các chương trình điều khiển thiết bị (driver) thực hiện. Ngoài ra còn có các giao diện lớp trên driver do hệ điều hành cung cấp. Các thành phần này nằm trong hệ thống vào ra của hệ điều hành. Một nhiệm vụ khác của hệ vào ra là tăng hiệu quả trao đổi thông tin với thiết bị ngoại vi nhờ hệ thống vùng đệm (buffer) và bộ nhớ cache. Như vậy, phân hệ quản lý vào ra của hệ điều hành gồm các thành phần sau:

- Các driver cho thiết bị cụ thể.
- Giao diện cho driver ở mức cao hơn.

Giới thiệu chung

- Mô đun quản lý vùng đệm, cache.

d. Quản lý file và thư mục

Để tránh cho người dùng không phải quan tâm tới đặc điểm các thiết bị nhớ ngoài vốn khác nhau và đa dạng, hệ điều hành cho phép sử dụng một khái niệm logic khi lưu trữ thông tin trên các thiết bị nhớ này, đó là file. File là tập hợp các thông tin có liên quan đến nhau, là nơi có thể ghi thông tin vào hoặc đọc thông tin ra. Các chương trình và người dùng không cần quan tâm tới việc file được cất giữ trên bộ nhớ ngoài như thế nào. Hệ điều hành sẽ chịu trách nhiệm ánh xạ file lên các thiết bị nhớ này.

Khi số lượng file lớn tới một mức nào đó, cần có cơ chế tổ chức các file sao cho dễ tìm kiếm và sử dụng. Chẳng hạn, nếu so sánh mỗi file như một quyển sách, khi số sách tương đối lớn như trong thư viện, người ta cần phân loại sách theo thể loại, tác giả .v.v. cho dễ tìm kiếm. Hệ điều hành phân chia các file thành các nhóm gọi là thư mục. Mỗi thư mục chứa các file có cùng một đặc điểm nào đó, ví dụ thư mục chứa các văn bản, thư mục chứa chương trình của cùng một hãng.

Hệ thống quản lý file và thư mục đảm nhiệm các chức năng sau:

- Tạo, xoá file và thư mục
- Đọc, ghi file
- Ánh xạ file và thư mục sang bộ nhớ ngoài

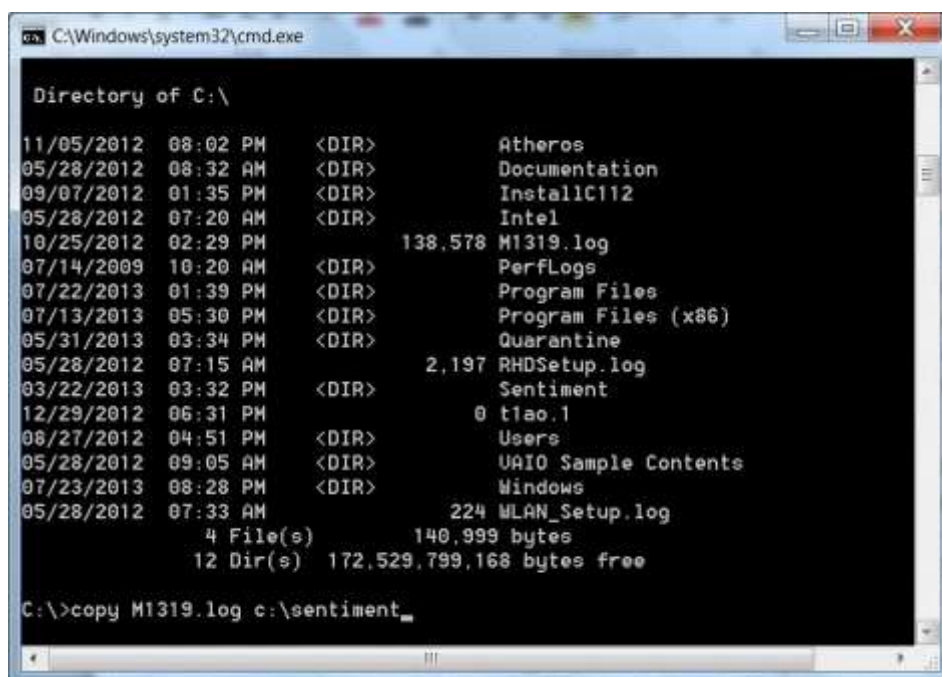
e. Hỗ trợ mạng và xử lý phân tán

Một trong các xu hướng phát triển của các hệ thống tính toán hiện nay là kết hợp máy tính vào các mạng máy tính. Điều này cho phép trao đổi, chia sẻ thông tin giữa các máy, đồng thời tạo khả năng xử lý phân tán. Các máy tính được nối với nhau qua các môi trường truyền thông cho phép truyền thông tin và dữ liệu. Đối với những bài toán lớn, đòi hỏi tốc độ tính toán cao hoặc khả năng lưu trữ dữ liệu lớn có thể phân tán việc xử lý trên các máy tính đã được nối mạng. Xử lý phân tán cho phép tận dụng tài nguyên của các máy riêng lẻ để tạo nên một hệ thống tính toán có khả năng lớn hơn nhiều.

Chức năng hỗ trợ mạng và xử lý phân tán của hệ điều hành bao gồm quản lý thiết bị mạng, hỗ trợ các giao thức truyền thông, quản lý việc truyền thông, cân bằng tải.

f. Giao diện với người dùng

Thành phần này được gọi bằng nhiều tên khác nhau như bộ dịch lệnh (command interpreter), vỏ (shell). Thực chất đây là giao diện giữa người dùng với hệ điều hành (cần phân biệt với các lời gọi hệ thống - system calls - là giao diện giữa các chương trình và hệ điều hành). Bộ dịch lệnh hay vỏ nhận lệnh từ người dùng và thực hiện các lệnh này, có thể bằng cách sử dụng dịch vụ do các phần khác của hệ điều hành cung cấp. Có thể lấy ví dụ các bộ dịch lệnh như **cmd.exe** của Windows, **bash** của Linux. Các lệnh được người dùng gõ trực tiếp dưới dạng văn bản. Số lượng lệnh có thể từ vài chục đến hàng trăm, từ những lệnh thông dụng như liệt kê thực mục (dir, ls), chép file (copy) tới những lệnh để thiết lập cấu hình mạng hoặc các thành phần khác của hệ thống. Trên hình 1.11 là ví dụ giao diện bộ dịch lệnh cmd.exe của Windows.



Hình 1.11. Bộ dịch lệnh cmd.exe của Windows

Trong các hệ điều hành hiện nay, bộ dịch lệnh thường được thay thế bằng các hệ giao diện đồ họa. Thay vì gõ các lệnh dưới dạng văn bản, người sử dụng làm việc với các đối tượng đồ họa như cửa sổ, biểu tượng rất trực giác và dễ hiểu. Các giao diện đồ họa thường được biết đến là **Windows Explorer** cho Windows (xem hình 1.12), **X windows** cho Linux.



Hình 1.12. Giao diện đồ họa của Windows 8

g. Các chương trình tiện ích và chương trình ứng dụng

Hệ điều hành thường chứa sẵn một số chương trình tiện ích và chương trình ứng dụng. Đây là thành phần không bắt buộc của hệ điều hành. Các chương trình tiện ích cung cấp cho người dùng một số dịch vụ giúp cho việc sử dụng hệ thống dễ dàng, hiệu quả hơn. Chẳng hạn có các tiện ích giúp nén tệp, chép các tệp dài ra đĩa mềm, tiện ích giúp lưu trữ dữ liệu.

Các chương trình ứng dụng hay có trong thành phần của hệ điều hành là các chương

trình dịch (trình dịch **Basic** của DOS, trình dịch C của Unix), các chương trình soạn thảo văn bản (**Notepad** của Windows, **vi** của Linux), các chương trình trò chơi. Các bản phân phối hệ điều hành Linux (Linux distributions) do các tổ chức khác nhau cung cấp như RedHat, Ubuntu chứa các chương trình tiện ích và ứng dụng tương đối khác nhau.

Trong nhiều trường hợp, việc xác định chương trình nào là tiện ích, chương trình nào thuộc hệ điều hành không đơn giản do không có tiêu chí rõ ràng để phân biệt.

1.7.2. Nhân của hệ điều hành

Nhân (kernel) là phần cốt lõi, là phần thực hiện các chức năng cơ bản nhất, quan trọng nhất của hệ điều hành và thường xuyên được giữ trong bộ nhớ.

Hệ điều hành là một hệ thống phức tạp, bao gồm nhiều thành phần, nhiều chương trình cấu thành. Vai trò của những thành phần rất khác nhau. Có những phần không thể thiếu, là cơ sở để cho toàn bộ hệ thống hoạt động, chẳng hạn như phần chịu trách nhiệm quản lý processor, quản lý bộ nhớ. Bên cạnh đó, nhiều chương trình thành phần của hệ điều hành cung cấp các chức năng kém quan trọng hơn. Các chương trình này có thể cần cho một số người dùng nhất định trong một số cấu hình nhất định, xong lại không cần cho người dùng khác trong các trường hợp khác. Ví dụ, một người sử dụng máy tính nghiệp dư sẽ không cần tới các chương trình dịch do hệ điều hành cung cấp. Hay một máy tính không nối mạng sẽ không bao giờ cần tới các dịch vụ mạng của hệ điều hành.

Từ nhận xét trên, thay vì tải toàn bộ hệ điều hành - có thể chiếm rất nhiều chỗ - vào và chứa thường xuyên trong bộ nhớ, người ta chỉ cần những thành phần quan trọng không thể thiếu được. Các phần này tạo thành nhân của hệ điều hành. Những phần còn lại không thuộc vào nhân có thể được tải vào và chạy khi cần thiết.

Chế độ nhân và chế độ người dùng. Thông thường, nhân của hệ điều hành được thực hiện trong *chế độ nhân*, hay còn gọi là *chế độ đặc quyền* (privilege mode), *chế độ hệ thống* (system mode). Trong khi đó, các chương trình khác như các trình ứng dụng được thực hiện trong *chế độ người dùng* (user mode), hay còn gọi là *chế độ bình thường* (normal mode). Máy tính hiện đại thường được thiết kế với hai chế độ thực hiện chương trình: *chế độ nhân* và *chế độ người dùng*. Chế độ nhân là chế độ mà chương trình thực hiện trong đó có đầy đủ quyền truy cập và điều khiển phần cứng máy tính, ví dụ có thể thay đổi nội dung tất cả các thanh ghi của CPU, hay có thể ghi vào bộ nhớ vật lý. Ngược lại, chương trình thực hiện trong chế độ người dùng bị hạn chế rất nhiều quyền truy cập và điều khiển phần cứng. Việc quy định chế độ cụ thể phụ thuộc vào một bit đặc biệt trong một thanh ghi của CPU: nếu bit này có giá trị bằng 0 thì chế độ là chế độ nhân, giá trị bit bằng 1 tương ứng với chế độ bình thường.

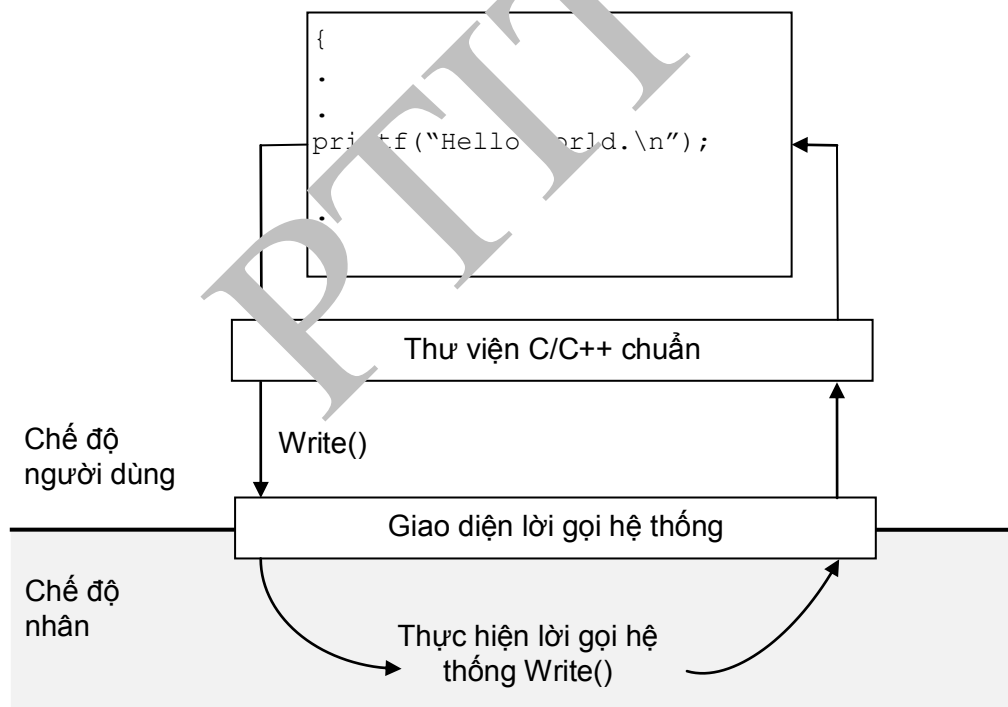
Việc phân biệt chế độ nhân và chế độ người dùng nhằm mục đích ngăn không cho chương trình ứng dụng vô tình hoặc cố ý thực hiện những thao tác làm ảnh hưởng tới hệ thống. Đây là điều kiện cần thiết để hệ thống máy tính hoạt động ổn định và hiệu quả.

Do nhân của hệ điều hành được thực hiện trong chế độ đặc quyền nên có toàn quyền kiểm soát và quản lý tài nguyên phần cứng. Trình ứng dụng phải thông qua nhân để có thể tiếp cận tài nguyên hệ thống. Ví dụ, khi cần thực hiện vào/ra thông tin, trình ứng dụng chạy trong chế độ người dùng không có quyền truy cập thiết bị ngoại vi để thực hiện vào ra. Thay

vào đó, trình ứng dụng yêu cầu hệ điều hành thực hiện thao tác vào/ra thông qua lời gọi hệ thống. Khi đó, hệ thống chuyển từ chế độ người dùng sang chế độ nhân, mô đun vào/ra của hệ điều hành thực hiện yêu cầu này và trả lại kết quả cho trình ứng dụng (xem hình 1.13). Do việc truy cập tài nguyên phải thông qua hệ điều hành nên các yêu cầu không hợp lệ sẽ bị phát hiện và ngăn chặn.

Một vấn đề đặt ra khi thiết kế hệ điều hành là quyết định phần nào của hệ điều hành thuộc vào nhân, phần nào không. Nói cách khác, nhân phải gồm những gì? Nhân càng lớn, càng gồm nhiều thành phần thì càng đảm đương được nhiều chức năng, và do vậy không cần gọi thêm các thành phần khác. Tuy nhiên, nhân lớn thì chiếm nhiều bộ nhớ, ảnh hưởng tới không gian nhớ dành cho các chương trình ứng dụng. Ngoài ra, tổ chức nhân lớn ảnh hưởng tới tính mềm dẻo, do việc thay đổi bất cứ thành phần nào của hệ điều hành đòi hỏi phải thay đổi nhân. Việc thay đổi, bổ sung các thành phần của hệ điều hành sẽ dễ dàng hơn nếu các thành phần được tổ chức dưới dạng các mô đun riêng lẻ (các chương trình) không thuộc nhân và chỉ chạy khi cần thiết. Do vậy, khi thiết kế hệ điều hành cần cân nhắc giữa các ưu nhược điểm của hai cách tổ chức: 1) nhân lớn, nhiều chức năng; và 2) nhân nhỏ, chuyển chức năng cho các mô đun khác.

Trong phần tiếp theo, ta sẽ xem xét một số cách tổ chức nhân và hệ điều hành.



Hình 1.13. Việc thực hiện lời gọi hệ thống dẫn tới chuyển từ chế độ người dùng sang chế độ nhân và ngược lại

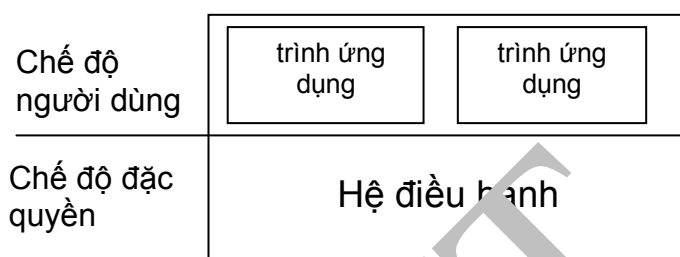
1.7.3. Một số kiểu cấu trúc hệ điều hành

Các thành phần đã nêu ở phần trên cần được tổ chức, kết hợp với nhau theo một cách nào đó để tạo ra một hệ thống thống nhất là hệ điều hành. Trong phần này, ta sẽ xem xét một số kiểu cấu trúc hệ điều hành thường được sử dụng.

1.7.3.1. Cấu trúc nguyên khối

Cấu trúc nguyên khối (monolithic), hay cấu trúc đơn giản, là cấu trúc trong đó toàn bộ các chương trình và dữ liệu của hệ điều hành có chung một không gian nhớ và do vậy có thể coi như một khối duy nhất. Hệ điều hành lúc đó trở thành một chương trình lớn, là tập hợp các thủ tục hay các chương trình con. Mỗi chương trình con có thể tự do gọi chương trình con khác khi cần thiết.

Cách tổ chức hệ điều hành như vậy cho ta hình ảnh tương tự với chương trình được viết theo kiểu lập trình cấu trúc, trong đó toàn bộ chương trình tạo thành từ các chương trình con, ví dụ chương trình viết trên ngôn ngữ C hay Pascal. Các chương trình con được dịch, sau đó liên kết thành một chương trình lớn. Việc che dấu thông tin hoàn toàn không có, tức là bất cứ chương trình con nào cũng có thể gọi chương trình con khác hoặc truy cập vào các dữ liệu chung của chương trình.



Hình 1.14 : Cấu trúc nguyên khối

Khi chương trình ứng dụng cần dùng tới các dịch vụ của hệ điều hành, chương trình ứng dụng sẽ sử dụng lời gọi hệ thống do hệ điều hành cung cấp. Lời gọi hệ thống được chuyển cho chương trình con tương ứng của hệ điều hành thực hiện. Chương trình con này sẽ gọi thêm các chương trình con khác để thực hiện nhiệm vụ nếu cần thiết.

Ưu điểm lớn nhất của cấu trúc nguyên khối là tốc độ thực hiện cao. Do có chung không gian địa chỉ, việc truy cập dữ liệu hoặc gọi chương trình con cần thiết được thực hiện nhanh chóng, không phải chịu những phí tổn về thời gian và bộ nhớ (ngăn xếp) như khi chuyển đổi giữa những mô đun có không gian nhớ khác nhau (giữa các chương trình khác nhau).

Nhược điểm của cách tổ chức này là thiếu tính mềm dẻo và khó làm cho hệ thống có độ tin cậy cao. Do toàn bộ nhân là một chương trình lớn nên việc thay đổi bất cứ thành phần nào cũng ảnh hưởng tới toàn bộ nhân. Điều này làm cho hệ thống không mềm dẻo, khó thay đổi. Ngoài ra, do toàn bộ hệ điều hành là một khối, mỗi chương trình con trong khối lại có thể truy cập tới dữ liệu và chương trình con khác, việc xuất hiện lỗi sẽ làm cho cả hệ thống tê liệt. Việc liên kết các thành phần thành một khối lớn còn khiến cho nhân luôn có kích thước lớn. Kể cả các thành phần không cần đến cũng được tải vào bộ nhớ cùng với các thành phần khác.

Để khắc phục các nhược điểm nói trên, trong một số hệ điều hành cấu trúc khối, người ta tổ chức các mô đun có thể tải từ đĩa và gắn vào nhân khi cần thiết. Trong trường hợp không dùng tới có thể xóa các mô đun khỏi nhân. Tuy là các mô đun riêng nhưng khi đã được tải vào bộ nhớ, các mô đun này nhập vào với nhân thành một khối có cùng không gian địa chỉ. Điều này đảm bảo cho hệ điều hành giữ được cấu trúc nguyên khối mà vẫn không có các thành